The Open Source Audit

A capstone project for OSS NGMC

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

STUDENT NAME	REGISTRATION NUMBER
Nitya N Gopalakrishnan	24BCY10250
SLOT	DATE OF SUBMISSION
B22	26.03.2026

Introduction

Linux, an open source operating system, was built by Linus Trovalds, in 1991.

Over the decades, it has expanded its user base to an extent that it is used in the most powerful supercomputers, in existence.

It is gravitated to for its versatility and secure database. The Linus kernel is maintained by a worldwide community of open source contributors and has a myriad of unique 'distros'.

Through the means of this documented report, we shall assimilate the technicalities associated with LINUX software and how the 21st century has used it to their advantage.

Chosen Software	LINUX Kernel

Part A: Origin and Philosophy(units 1&2)

A1. The problem this software was created to solve

Precursor of LINUX- UNIX:

Dennis Ritchie and Ken Thompson were the first to design an operating system named ,Unix ,in 1969. Unix was started as a small project at AT & T Bell Lab.

It had 7 versions which couldn't withstand the growing competition among supercomputers and was soon taken over by MINIX- a lightweight UNIX-inspired operating system created by Professor Andrew Tanenbaum. Unfortunately, MINIX too, fell short, as it couldn't take full advantage of the powerful new 386 processors.

Beginning of Linux-

Linus Benedict Torvalds, while studying at the University of Helsinki in Finland in 1991 , was using Minix, a non-free OS. At that moment, he became aware of the need for a free and open-source operating system that would forward the goals of GNU.

Linus did not hail from a very flourishing family and thus exercised his tech expertise on a Sinclair QL.

When he finally managed to buy his first PC, thanks to a fortunate drop ion hardware prices and a lucky turn of personal events, he as usual, sat down to code. However, he was unable to plunge into the world of Unix immediately, since it took several months for the MINIX floppy disks to turn up.

Motivated by the need for an open-source, customizable OS, Torvalds released the kernel under the GNU GPL, leveraging UNIX's design. The early versions of Linux focused on kernel stability, with major releases like 1.0 in 1994 adding ne tworking support.

Initially, the operating system just had free access, user driven customization, a GNU GPL which ensured sharing and a community. Every Linux-based OS includes the Linux kernel-which manages hardware resources—and a set of software packages that make up the rest of the operating system.

In 1993, Slackware Linux, one of the oldest and most traditional Linux distributions, was released. It provided a simple and customi zable way to install and use Linux, making it popular among early Linux enthusiasts. Three years hence, Debian came into existence. It introduced the concept of a large and well-organized software repository, which made it easier to install and manage software packages. Following this was the release of Red Hat Linus 6.0, in 1999. Red Hat Linux 6.0 was a significant release that brought Linux into the mainstream enterprise market. It offered better hardware support, improved security, and a user-friendly installation process. Ubuntu, a Debian-based Linux distribution, was launched. It aimed to provide an easy-to-use and user-friendly Linux experience, targeting both desktop and server users. Ubuntu's regular release cycles and long-term support (LTS) versions made it very popular.

Android, an operating system based on the Linux kernel, became the most widely used mobile operating system in the world. This demonstrated the versatility and scalability of the Linux kernel.

A2. The license — what it actually says

- The 4 freedoms of free software

  Richard Stallman and the Free Software Foundation (FSF) came up with the idea of the Four Freedoms. To be considered "Free," a license must allow:

  Freedom 0: You can run the program however you want and for whatever reason you want.
  Freedom 1: You have the right to learn how the program works and change it so that it does what you want it to do.
  Freedom 2: The freedom to give away copies to help other people.
  Freedom 3: You can give copies of your changed versions to other people.

  In case of Linux, these rights are clearly protected by the GPLv2. It makes sure that anyone who gets a copy of the software can still get to the source code.

- Can a company take this software, modify it, and sell it without sharing their changes? Why or why not?

  No. This is because of something called "Copyleft." If a company changes the Linux kernel and then shares that version (by selling it or giving it away), they must give the people who get it the source code of their changes under the same GPLv2 license.
  The "Sharing" Part: If you share the binary, you can't keep the improvements a secret. This stops "proprietary hijack ing" of the code. Though, you can still SELL free software.

- What is the difference between GPL and MIT licensing? Which would you choose for something you built, and why?
  GPL ensures software stays free forever. While, MIT makes the software as easy to use.

Derivative works must be licensed under the GPL while for MIT, it can be kept private until turned into a proprietary software.

For a small library or a tool that I want as many people as possible to use, I'd pick MIT. I want people to be able to use it anywhere, even in closed-source commercial apps, without any legal problems.

For a big project that lots of people will work on together, like an operating system, I would choose GPL. This way, if a big company makes my work better, those improvements will help everyone instead of becoming a trade secret.

- Is there a difference between 'free as in free beer' and 'free as in freedom'? Give an example from your chosen software.

A3. The Ethics of Open-Source

Open source is not just about licensing, it is also about how knowledge should be shared among people. Linux kernel is a very good example of this coz it has been developed by many people around the world over time and keeps improving.

- Should all software be open source?

On one side, open source software has many benefits. When the code is open, anyone can see it, fix bugs and even improve it. This makes the software more secure and reliable in long term. Linux kernel shows how powerful this idea is, since dit is used in servers, phones and many systems. Also it reduces cost and helps students to learn from real code.

On other side, not all software should be open source. Companies invest a lot of mon ey and time in making software, so they need profit also. If everything becomes open source, companies may not get enough return. Also sometimes open source software is difficult to use for normal users & requires technical knowledge. So keeping both types is more practical.

- Is it ethical for companies to profit from free community work?

Companies like Red Hat, Google and Meta use Linux kernel in their systems.

In some ways, it ethical because these companies also contribute back. For example Red Hat provides enterprise support and also improves Linux. Google and Meta also give updates & patches. Since open source allows this usage, they are not doing anything wrong.

But there is also another side. Some companies may earn huge profits but contribute very less. This can feel unfair because many developers wo rk for free or for learning. If companies don’t give bac k enough, it might reduce motiva tion of contributors.

- “Standing on the shoulders of giants” in software

This phrase means building new things using already existing work. In software, this happens a lot. Linux kernel itself is base for many technologies today.

Open source helps innovation because developers don't have to start from 0. They can directly improve existing systems and make new features. This saves time and increases progress

But some people think it reduces originality, because people depend too much on old systems. Still most innovations are step by step so open source actually helps more than it harm

- Overall Reflection

The Linux kernel shows both good and challenging sides of open source. It supports collaboration and fast development, but also raises questions about fairness between companies and community.

Overall, open source increases innovation and sharing of knowledge, but it is important that companies and developers maintain a balance in contribution and benefits.

Open source accelerates innovation by providing strong foundation. Developers do not need to start from scratch''they can improve existing systems, leading to faster technological progress. The Linux kernel is a prime example, powering everything from smartphones to supercomputers.
Some argue it may reduc origin ality, as developers rely heavily on existing frameworks instead of creating entirely new systems. However, most innovations are incremental, &open source supports this natural evolution.

Part B-Linux Footprint(Unit 2)

This report mentions the methodology of installation in an Ubuntu ecosystem.

• Installation method — is it available via RPM, apt, or must it be compiled from source?

The kernel is rarely compiled from source by the average user.

The majority of users install pre-compiled binary packages using package managers (apt/dnf). ***apt*** is used on Ubuntu to install packages such as ***linux-image-generic*.**

Compilation from Source: Developers typically use make and make install to manually compile the source after downloading it from kernel.org and configuring it using make ***menuconfig*.**

Update Command: The distribution maintainers' most recent stable kernel image is pulled by running ***sudo apt update && sudo apt*** upgrade.

• Key directories — where are its binaries, config files, and logs stored? (e.g., /usr/bin, /etc, /var/log)

1. Boot Directory(/boot): It contains files needed to start the OS before the filesystem is mounted.
   /boot/vmlinuz-$(uname -r): It is a compresses executable file that the bootloader loads into the memory to start the OS.
   /boot/initrd.img-$(uname -r): It contains a tiny temporary filesystem with the necessary drivers that is needed to finish the boot process.
   /boot/config-$(uname -r): A text file listing every option and driver that was enabled or disabled when the kernel was compiled. Good for troubleshooting incompatibility of hardware.

2. Modules Directory: The kernel can't possibly keep every driver for all hardware ever made—otherwise, the file would be enormous. So, it uses modules instead.
   Inside ***/lib/modules/$(uname -r)/,*** you'll find folders for different types of drivers, like network, sound, and video. These are **.ko** files (kernel objects), and the kernel loads them only when you need them. If you plug in something new, the kernel digs through this directory to find the right code to handle it.

3. Virtual Filesystems: They aren't actual folders stored on your disk.
   /proc gives information on running processes and system resources.
   For example, ***/proc/cpuinfo*** tells you about your processor and ***/proc/meminfo*** shows your RAM details.
   /sys is for tinkering with hardware settings. You can change things like screen brightness or disk power states just by writing values to files here.
4. Device Directory: ***/dev-*** kernel handles the hardware, but represents it as files in this directory. So your hard drive might be ***/dev/sda***.
5. Finally, logging and debugging live in ***/var/log*** files or running ***journalctl -k*** ,show messages from the kernel. When hardware fails or a driver crashes, the kernel dumps error details here. If your computer starts acting weird, system administrators check these logs first.

• User and group — what Linux user and group does it run under? Why does this matter for security?

The kernel isn't like an app—you don't run it as a "user." It lives in Kernel Space and pretty much calls the shots for the hardware. The stuff you normally do, those processes, run in User Space under the kernel's watch. When it comes to root (UID 0), that's the main element. Most jobs that tap directly into the kernel get started by root.

If you try to snoop in someone else's files, the kernel checks your permissions and shuts you down if you don't have access. If there's a security hole in the kernel, it's a big deal—hackers can pull off something called "privilege escalation."

- Service management — can it be started, stopped, or restarted as a service? Show the commands.
    1. Check status- ***lsmod***
    2. Start/load a feature- ***sudo modprobe <module_name>***
    3. Stop/remove a feature: ***sudo modprobe -r <module_name>***
    4. Boot Management: GRUB is used to select the kernel version.

• Update model — how does the open-source community push security patches to your machine?

1. Upstream: A bug is found and a patch is passed to LKML
2. Distribution maintainers: Teams fix it and backport to the specific version of the kernel,shipped to the user.
3. Push to Repositories: The patched kernel is placed in the official package repository.
4. Installation: On upgrading, the new kernel is downloaded to bootloader.
5. Activation: The update requires the system to be rebooted, so as to stop the old kernel.

Part C- The FOSS Ecosystem(Units 3 &4)

The Linux kernel isn't just a piece of code; it has a vast ecosystem, running in the background which requires to be understood. From how it is built, what changes it can induce , to how it is governed.

- Its Core Dependencies(tools it is built on):
  The Linux kernel can build itself, i.e, it's self-hosting. However, to actually turn all that source code into something that boots up, it needs these open source tools, specific ones mostly.

  It's written in C for the most part, with bits of assembly thrown in, with newly added Rust too. So, compilers like GCC from the GNU or LLVM Clang are what it depends on a lot. Without them, nothing gets compiled right.

  Then there's the build side, using GNU Make to handle everything. Millions of lines of code, that sounds messy to manage, is kept track of.

  Git is how they control versions for the kernel. Linus Torvalds made Git himself because other tools were too slow or couldn't deal with how big the project got.

  The kernel runs on its own, but it has this system call interface that GLIBC uses, the GNU C Library. GLIBC connects pretty much every app to the kernel, acting like a bridge. Without it, applications wouldn't know how to talk to the system at all.

- Inspirations and Spin-offs:
  The Linux kernel has done really well, and that has led to all sorts of new ideas in the industry, including some big changes or forks from the original.

  Android stands out as the biggest example. It takes a tweaked version of the kernel to manage hardware and stuff like memory. But then it goes in a whole different direction for the user side, not like what you see on a regular desktop setup such as Ubuntu. That makes sense for phones, I guess, since they need something more mobile friendly.

  Then there's this real time version, PREEMPT_RT. Its meant for things where timing is super important, like robots in factories or medical devices. You need everything to happen right down to the microsecond, otherwise problems could come up.

  Virtualization is another area the kernel got into with KVM, which is Kernel based Virtual Machine. Basically, it lets Linux act as a hypervisor, and that powers a lot of the cloud stuff today. Places like AWS or Google Cloud can run tons of virtual servers all on Linux underneath. Its kind of wild how that scaled up so fast.

  Some projects got inspired by Linux but arent direct forks. ReactOS tries to clone Windows in open source, and there are hobbyist kernels people mess around with. They draw from that collaborative way Linux works, the Bazaar model or whatever its called. It feels like that approach just encourages more people to build on it, though not everything turns out perfect.

- The Web and LAMP stack:
  The 'L' in LAMP stands for Linux.
  L=Linux(OS)
  A=Apache(web server)
  M=MySQL(database)
  P=Php/python/pearl(languages)

  Linus provides a stable and efficient foundation. Since it handles networking protocols efficiently, it has become the default choice for users..

- Community and Governance
  Linux doesn't have any owner but it does follow a hierarchy.
  The Linux foundation funds the projects but doesn't contribute technically, i.e isn't involved in coding.
  Linus Torvalds- the creator of Linux, has the final say on what code enters the kernel. He intructs the subsystem maintainers.

  Apart from these, the communication is done via channels like LKML, IRC and conferences.

Part D-Open Source vs Proprietary

Dimension	Open Source	Proprietary Alternative
Cost	Free to use and modify. No licensing fee	Requires paid license. Different versions have different costs.
Security(who can audit the code?)	Publicly available source code, can be audited by anyone	Source code ius closed; cqan be audited by Microsoft developers only.
Support and Reliability	Community driven support	Professional customer support from Microsoft.
Freedom to Modify	Full freedom to modify & distribute kernels.	Restricted access to source code; modification not permitted.
Community vs corporate control	maintained by a global community of developers.Decisions are collaborative.	Centralised decisions by Microsoft.
Your overall verdict	Ideal for developers and companies needing customization frequently	Best for usability and compatibility. Suitable for non-professionals

The Linux kernel stands out for its transparency flexibility & cost-effectiveness. Its open source nature allows developers worldwide to inspectand improve the system, making it highly secure and reliable, especially in server environments. Organisations that require customization, scalability & control over their systems often prefer Linux-based OS. However, it may require technical expertise to manage and configure effectively.

On the other hand Microsoft Windows provides a more user-friendly experience with strong commercial support and broad software compatibility. It is ideal for general users and businesses that prioritise ease of use over customization. For real-wor ld deployment, I would recommend the Linux kernel in environments like servers, cloud sy stems, and development platforms, while Windows remains a strong choice for desktop users. Yes, I would be willing to contribute to the Linux ecosystem, as its open-source model encourages learning, collaboration and innovation.

Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Code:

```
#!/bin/bash

# Script 1: System Identity Report

# Author: [your name]

# Description: [displays system details like kernel,user,uptime,etc]

# static identifying info

student_name="Nitya N Gopalakrishnan"

software="Linus OS"

kernel=$(uname -r)          # releases kernel version

user_name=$(whoami)    # returns username of the coder

uptime=$(uptime -p)       # shows the duration of the system's running state

DATE_Time=$(date)    # displays system's current date and time

distro=$(cat /etc/os-release | grep PRETTY_name | cut -d= -f2 | tr -d '"')


echo " System Identity Report"

echo "----------------------"

echo "Student name : $student_name"

echo "user : $user_name"

echo "home directory : $HOME"

echo "kernel : $kernel"

echo "distribution : $distro"

echo "uptime : $uptime"

echo "Date and Time : $DATE_Time"

echo "License : This OS is covered under GPL "
```

1. Shebang(#!): **#!/bin/bash** tells the OS which interpreter to use, ensuring that the script runs as a Bash script instead of a different shell.
2. Scalar variables are used to store both strings and dynamic data.
3. **$(..):** This allows the script to run as a Linus command and capture its output directly in a variable.
4. **(|) Pipelining:** This is used to pass data from one tool to the other- **grep(**used for filtering), **cut**(used for splitting) and **tr**(used for removing quotes).
5. **Environment variables** line **$HOME** are predefined by Linux OS to track the user's directory.
6. **Echo** is used to print the output in the terminal and display the stored data.

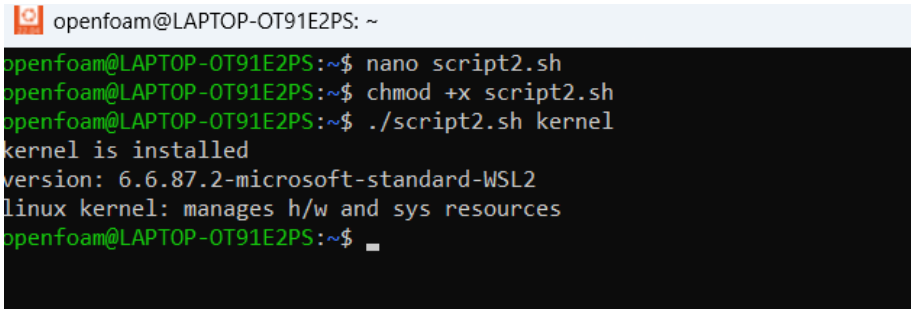
```
openfoam@LAPTOP-OT91E2PS: ~
  Usage of /:   0.4% of 1006.85GB   Users logged in:       0
  Memory usage: 10%                 IPv4 address for eth0: 192.168.74.149
  Swap usage:   0%


This message is shown once a day. To disable it please create the
/home/openfoam/.hushlogin file.
openfoam@LAPTOP-OT91E2PS:~$ nano script1.sh
openfoam@LAPTOP-OT91E2PS:~$ chmod +x script1.sh
openfoam@LAPTOP-OT91E2PS:~$ ./script1.sh
 System Identity Report
----------------------
Student name : Nitya N Gopalakrishnan
user : openfoam
home directory : /home/openfoam
kernel : 6.6.87.2-microsoft-standard-WSL2
distribution :
uptime : up 11 minutes
Date and Time : Sun Mar 22 11:10:55 IST 2026
License : This OS is covered under GPL
openfoam@LAPTOP-OT91E2PS:~$
```

Script 2 — FOSS Package Inspector (Unit 2)

Code

```
#!/bin/bash

# Script 2: FOSS Package Inspector

# Author: [User's name]


#taking input

PACKAGE=$1


#check if the input is given

if [ -z "$PACKAGE" ]; then

        echo "please give a packge name"

        exit 1

fi

#handling Linux kernel

if [ "$PACKAGE" == "kernel" ]; then

        echo "kernel is installed"

        # display version

        version=$(uname -r)

        echo "version: $version"

else
```

```
# checking using dpkg
        if dpkg -l | grep -q "^ii $PACKAGE"; then
                echo "$PACKAGE installed"
                dpkg -l | grep "$PACKAGE"
        else
                echo "$PACKAGE not installed"
        fi
fi
# test case for description
case $PACKAGE in
        kernel)
      echo "linux kernel: manages h/w and sys resources"
          ;;
        firefox)
          echo "Firefox: open source browser"
          ;;
        vlc)
          echo "vlc: media player"
          ;;
        *)
           echo "nothing here"
           ;;
Esac
```



The script checks whether the selected software, Linus kernel, is present in the system ad if so, displays its version. The script uses **uname -r** command to retrieve the version since the kernel is not a regular installable package. For the verification of other packages, commands like **dpkg** and **grep** are used along with piping. Command line arguments are also used to fetch user inputs along with **if-else** statements for decision making. Standard **echo, $(), case** statements are used like used for the same role in the previous code.

Script3-Disk and Permission Auditor

Code:

```
#!/bin/bash
# script3: disk and permission auditor
# it checks some imp system directories and prints permissions,
# and disk usage
# list of directories
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
echo "directory audit report"
echo "----------------------"

#looping thru each dir
for DIR in "${DIRS[@]}"; do
	# if dir exists:
	if [ -d "$DIR" ]; then
	#get permissions
	# awk is used to extract needed cols
		PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
	#get disk usage of directory
	# du -sh gives size
		SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
	#print result
		echo "$DIR => Permissions: $PERMS | Size: $SIZE"
	else
		echo "$DIR does not exist"
	fi
done
#checking config directory
CONFIG="/etc"
#checking existence
if [ -d "$CONFIG" ]; then
	CONF_PERM=$(ls -ld $CONFIG | awk '{print $1,$3}'
	echo "Config dir ($CONFIG) permissions are: $CONF_PERM"
else
	echo "dir not found"
fi
```

```
openfoam@LAPTOP-OT91E2PS:~$ nano script3.sh
openfoam@LAPTOP-OT91E2PS:~$ chmod +x script3.sh
openfoam@LAPTOP-OT91E2PS:~$ ./script3.sh kernel
directory audit report
---------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxr-xr-x root syslog | Size: 195M
/home => Permissions: drwxr-xr-x root root | Size: 96K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 278M
/tmp => Permissions: drwxrwxrwt root root | Size: 20K
```

This script analyzes the system's disk usage and checks file permissions in a specified directory. It displays information about total disk space, used and available space, and lists files with their permissions. The script helps users understand storage usage and identify files with restricted or open permissions. It uses standard Linux commands to gather and present this information in a readable format.

Script4-Log File Analyzer

Code:

```
#!/bin/bash

# script4: log file analyser

# author: Nitya Gopalakrishnan

#description: counts occurences of a keyword in a log file

#taking command like arguments


LOGFILE=$1

KEYWORD=${2:-error} #default

#check if file is provided

if [ -z "$LOGFILE" ]; then

        echo "Usage: $0 <logfile> [keyword]"

        exit 1

fi

#check if file exists

if [ ! -f "$LOGFILE" ]; then

        echo "Error: File $LOGFILE not found"

        exit 1

fi


#initialise counter

COUNT=0

#read file line by line

while IFS= read -r LINE; do

        #check if line has keyword

        if echo "$LINE" | grep -iq "$KEYWORD"; then

                COUNT=$((COUNT + 1))

        fi

done < "$LOGFILE"
```

```
#results

echo "File : $LOGFILE"

echo "Keyword : $KEYWORD"

echo "occurences : $COUNT"


#last 5 matching lines

echo "Last 5 lines:"

grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

```
openfoam@LAPTOP-OT91E2PS:~$ nano script4.sh
openfoam@LAPTOP-OT91E2PS:~$ chmod +x script4.sh
openfoam@LAPTOP-OT91E2PS:~$ ./script4.sh kernel
Error: File kernel not found
```

This script reads a log file and counts how many times a specific keyword (like “error”) appears. It processes the file line by line and displays the total count along with matching lines.

Dependencies- 1.grep 2.cat 3.tail

Script5-Open Source Manifesto Generator

Code:

```
#!/bin/bash

# Script 5: OS Manifesto Generator

# Author: Nitya Gopalakrishnan

# Description: generates a personalised open source manifesto and saves it to a file

# alias ll='ls -l'


echo "answer 3 ques to generate your manifesto:"

echo ""


# taking user input

read -p "1. Name one open source tool you use daily: "TOOL

read -p "2. In one word, what does 'freedom' mean to you? "FREEDOM

read -p "3. Name one thing you would build and share freely: "BUILD


# get current date

DATE=$(date "+%d %B %Y")


#output file name

OUTPUT="manifesto_$(whoami).txt"
```

```
echo "open source manifesto--" > "$OUTPUT"

echo "Date: $DATE" >> "$OUTPUT"

echo "" >> "$OUTPUT"


#creating paragraph

echo "open source software is quite handy. Using tools like $TOOL, one can
experience $FREEDOM. It inspires one to create projects like $BUILD . Thus, it
promotes transparency and is free." >> "$OUTPUT"

echo "" >> "$OUTPUT"

echo "---" >> "$OUTPUT"


echo "manifesto saved to $OUTPUT"

cat "$OUTPUT"
```

```
openfoam@LAPTOP-OT91E2PS:~$ nano script5.sh
openfoam@LAPTOP-OT91E2PS:~$ chmod +x script5.sh
openfoam@LAPTOP-OT91E2PS:~$ ./script5.sh
answer 3 ques to generate your manifesto:

1. Name one open source tool you use daily: TOOL Linux
2. In one word, what does 'freedom' mean to you? FREEDOM Choice
3. Name one thing you would build and share freely: BUILD ai chatbot
manifesto saved to manifesto_openfoam.txt
open source manifesto--
Date: 26 March 2026

open source software is quite handy. Using tools like , one can experience . It inspires one to create projects like
Thus, it promotes transparency and is free.
```

This script generates a personalized open-source manifesto by asking the user three questions. It stores the responses, combines them into a paragraph, adds the current date, and saves the output into a text file.

Dependencies- 1.read(in-built) 2.date 3.echo 4.cat